

Documentación Técnica: Automatización de Despliegue Fail2ban (Entorno PRAN PRE)

1. Objetivo del Despliegue

Este documento detalla el funcionamiento del playbook `Fail2ban_pre_pran.yml`. El objetivo es automatizar y estandarizar la instalación de Fail2ban y el envío de alertas por correo (MSMTP) en el entorno de preproducción. El proceso está diseñado para evitar configuraciones manuales por SSH y asegurar que todos los servidores queden idénticos.

2. Variables Principales

El código utiliza variables para que sea fácilmente escalable a otros entornos sin tener que modificar las tareas principales:

- `usar_msmtplib`: Variable (`true/false`) para decidir si instalamos el cliente de correo. Se marca como *false* si el servidor ya tiene un servicio de correo propio funcionando para evitar pisar configuraciones.
- `dir_files_base` y `entorno_conf`: Rutas donde centralizamos los archivos de configuración en el servidor de Ansible.

3. Fases de Ejecución

Fase I: Comprobación del entorno (Tareas 0a - 0b)

El playbook revisa primero qué tipo de servidor web está corriendo. Comprueba si existe la ruta de instalación de OpenResty y guarda el resultado en la variable `tipo_web` (Nginx u OpenResty). Esto sirve para inyectar automáticamente los archivos de configuración correctos más adelante.

```
tasks:
  - name: 0a. Comprobar si el servidor utiliza OpenResty
    ansible.builtin.stat:
      path: /usr/local/openresty/nginx/sbin/nginx
      register: openresty_bin

  - name: 0b. Definir variable del motor web
    ansible.builtin.set_fact:
      tipo_web: "{{ 'openresty' if openresty_bin.stat.exists else 'nginx' }}"
```

Fase II: Instalación y arranque inicial (Tareas 1 - 2c)

Se instala el repositorio EPEL y la versión más reciente de Fail2ban. Aquí aplicamos una buena práctica de despliegue: en lugar de crear archivos de log a mano, arrancamos el servicio de Fail2ban con su configuración por defecto. Esto hace que el propio sistema genere su log (/var/log/fail2ban.log) con los permisos correctos, algo totalmente necesario para que luego la regla recidive funcione sin dar fallos de lectura. Por último, se instala MSMTTP si la variable inicial lo permite.

```
- name: 1. Instalar repositorio EPEL
  ansible.builtin.dnf:
    name: epel-release
    state: present

- name: 2a. Instalar paquete Fail2ban
  ansible.builtin.dnf:
    name: fail2ban
    state: latest
    update_cache: true

- name: 2b. Arrancar Fail2ban con config por defecto (crea el log de forma nativa)
  ansible.builtin.service:
    name: fail2ban
    state: started
    enabled: true

- name: 2c. Instalar MSMTTP (solo si el servidor no tiene correo nativo)
  ansible.builtin.dnf:
    name: msmtpp
    state: latest
  when: usar_msmtpp
```

Fase III: Copia de configuraciones (Tareas 3 - 4b)

Copiamos nuestros filtros de bloqueo corporativos (errores SSL, peticiones 4xx, accesos denegados) a la carpeta /etc/fail2ban/. También pasamos la configuración del jail.local definitivo y las rutas adaptadas al servidor web que detectamos en la Fase I. Todo se copia asegurando permisos restrictivos (0644) y propiedad para root.

```
- name: 3. Desplegar filtros comunes
  ansible.builtin.copy:
    src: "{{ dir_files_base }}/Instalacion/{{ item.src }}"
    dest: "/etc/fail2ban/{{ item.dest }}"
    owner: root
    group: root
    mode: '0644'
  loop:
    - { src: 'nginx-sslerror.conf', dest: 'filter.d/nginx-sslerror.conf' }
    - { src: 'nginx-4xx.conf', dest: 'filter.d/nginx-4xx.conf' }
    - { src: 'nginx-forbiden.conf', dest: 'filter.d/nginx-forbiden.conf' }
    - { src: 'sendmail-whois-lines.local', dest: 'action.d/sendmail-whois-lines.local' }

- name: 4a. Desplegar archivos de configuracion especificos de PRAN PRE
  ansible.builtin.copy:
    src: "{{ dir_files_base }}/{{ entorno_conf }}/{{ item.src }}"
    dest: "/etc/fail2ban/{{ item.dest }}"
    owner: root
    group: root
    mode: '0644'
  loop:
    - { src: 'jail.local', dest: 'jail.local' }
    # En la siguiente línea: usa la variable tipo_web para elegir el origen
    - { src: 'paths-overrides-{{ tipo_web }}.local', dest: 'paths-overrides.local' }
    - { src: 'sendmail-common.local', dest: 'action.d/sendmail-common.local' }
```

```
- name: 4b. Desplegar configuracion de MSMTMP (solo si usar_msmtmp es true)
  ansible.builtin.copy:
    src: "{{ dir_files_base }}/{{ entorno_conf }}/msmtprc"
    dest: /etc/msmtprc
    owner: root
    group: root
    mode: '0644'
  when: usar_msmtmp
```

Fase IV: Aplicación de cambios (Tarea 5)

Activamos Fail2ban para que arranque siempre con el sistema operativo y hacemos un reinicio del servicio. Como el archivo de log ya se creó de forma natural en la Fase II, el servicio levanta a la primera y aplica todas nuestras reglas restrictivas.

```
- name: 5. Habilitar y reiniciar el servicio Fail2ban
  ansible.builtin.service:
    name: fail2ban
    enabled: true
    state: restarted
```

4. Ventajas del Modelo

- **Ejecución segura:** El playbook se puede lanzar repetidas veces sobre la misma máquina sin romper nada. Si el servidor ya está bien configurado, Ansible lo detecta y no aplica cambios.
- **Evita errores humanos:** Al dejar que el servicio genere sus propios archivos de control en el arranque inicial, eliminamos la necesidad de crear carpetas o parches manuales previos.
- **Escalable:** Gracias a la detección automática en el primer paso, usamos exactamente el mismo código para servidores puros de Nginx o para balanceadores con OpenResty.